Jira, GitHub, Scrum, and Submission Guidelines

Working Rules and Evidence Standards

Module	Software Architecture and Process Models
Student Level	Level 2
Assignment Duration	7 weeks
Document	06 of 07
Jira Structure	One Jira project per seven-member team
GitHub Structure	One GitHub organization per overall project

1. Jira Project Setup

Each team must create and maintain its own Jira project. This keeps membership within the free-plan limit while preserving team-level ownership. The team should add the seven students and reserve any remaining access for teaching or project coordination as instructed.

Team	Recommended Jira Project Key	Project
Group 1	AMS-G1	Apartment - Identity and Resident Management
Group 2	AMS-G2	Apartment - Property, Lease, and Occupancy
Group 3	AMS-G3	Apartment - Billing, Utilities, and Payments
Group 4	AMS-G4	Apartment - Operations and Community
Group 5	USM-G5	University - Identity and University Directory
Group 6	USM-G6	University - Facilities and Reservations
Group 7	USM-G7	University - Service Requests and Work Orders
Group 8	USM-G8	University - Events, Communications, and Feedback

2. Jira Issue Hierarchy and Required Fields

Issue Type	Use	Required Content
Epic	Major business capability or release objective.	Purpose, scope, owner, linked requirements, and completion condition.
Story	User-valued behavior.	User story statement, acceptance criteria, priority, estimate, dependency links, owner, and test evidence.
Task	Technical, design, documentation, deployment, or process work.	Clear outcome, owner, estimate, linked story/epic, and evidence.
Subtask	A small part of a story or task.	Specific action, owner, and done condition.
Bug	Observed deviation from expected behavior.	Environment, steps, expected result, actual result, severity, evidence, owner, fix version, and retest result.
Change Request	Approved scope or contract change.	Reason, impact, decision, approver, affected stories/APIs, and implementation plan.

3. User Story and Acceptance Criteria Standard

> **User story format**
> As a [role], I want [capability], so that [benefit].

Acceptance criteria should be testable. Teams may use bullet conditions or Given-When-Then format.

> **Acceptance criteria format (Given-When-Then example)**
> Given an eligible user and an available facility, when the user submits a valid reservation request, then the reservation is created with Pending status and becomes visible to the resource manager.

- Avoid stories such as "Create backend" or "Do frontend" because they do not describe user value.
- Split stories that cannot be completed, tested, and reviewed within the selected sprint.
- Link technical tasks and tests to the story they support.
- Add dependency links or explicit references when another team API or frontend change is required.

4. Recommended Workflow

Status	Meaning
Backlog	Defined but not committed to the current sprint or iteration.
Ready	Meets the Definition of Ready and can be selected for implementation.
In Progress	Actively being worked on by an assigned member.
Code Review	Implementation is in a pull request and awaiting review or correction.
QA / Verification	Merged or deployable work is being tested against acceptance criteria.
Done	Meets the Definition of Done and has linked evidence.
Blocked	Cannot progress; blocker, dependency, owner, and next action are recorded.

5. Sprint and Scrum Guidance

Teams may choose their sprint length. A one-week or two-week cadence is recommended. The chosen cadence must be justified in the project plan and used consistently unless a documented retrospective supports a change.

- Complete at least two time-boxed development iterations before the final release and closure activities.
- Conduct sprint planning, short progress coordination meetings, sprint review, and retrospective.
- Keep the sprint goal visible in Jira and do not overload the sprint with unestimated work.
- Move unfinished items back to the backlog or into a justified next sprint; do not mark incomplete work Done.
- Use a final stabilization window for integration, defect correction, deployment, documentation, and demonstration preparation.

6. Definition of Ready and Definition of Done

Definition of Ready	Definition of Done
Purpose and user value are clear.	Acceptance criteria pass and evidence is linked.
Acceptance criteria are testable.	Code is reviewed and merged through a pull request.
Dependencies and API assumptions are identified.	Automated and relevant manual tests pass.
Design or contract information is sufficient.	Swagger/API docs and README are updated.
Estimate and owner are assigned.	No secrets or prohibited data are committed.
The item can reasonably finish within the sprint.	The integrated environment is not broken and Jira status is updated.

7. Cross-Team Synchronization with Separate Jira Projects

- Every team appoints one Team Lead.
- The four team leaders for a project meet weekly and maintain shared integration notes in the GitHub documentation repository.
- Breaking API changes require a documented change request and agreement before merge.
- Shared milestone dates, API freeze, release candidate, and final release must be aligned across the four Jira projects.
- Each team remains responsible for testing its behavior in the complete system.

8. GitHub Organization and Repository Structure

Organization	Required Repositories
Apartment project organization	shared-frontend; api-gateway; identity-access-service; resident-management-service; property-unit-service; lease-occupancy-service; billing-payment-service; utility-charge-service; operations-service; community-service; deployment-infrastructure; project-docs.
University project organization	shared-frontend; api-gateway; identity-access-service; campus-directory-service; facility-resource-service; reservation-service; service-request-service; work-order-service; event-service; communication-feedback-service; deployment-infrastructure; project-docs.

The exact organization and repository names may be shortened, but the ownership and purpose must remain clear. Repositories must not be duplicated merely to avoid collaborative integration.

9. Branching and Pull-Request Rules

- Protect the main branch. Direct pushes to main are not permitted.
- Use feature branches such as feature/AMS-G3-invoice-generation or fix branches such as fix/USM-G7-status-validation.
- Open a pull request with purpose, linked Jira issue, screenshots or API evidence when relevant, testing performed, and known limitations.
- At least one meaningful peer review is required before merge. Shared frontend, gateway, security, or integration changes should be reviewed by a relevant liaison or owner.
- Resolve review comments or record a justified decision before merge.
- Keep pull requests focused. Large changes should be split when practical.
- Maintain a clear release tag history.

10. Commit and Traceability Standard

- Use concise commits that represent a coherent change.
- Reference the Jira key in commit messages or pull-request titles, for example: AMS-G3-24 add invoice balance calculation.
- Do not combine unrelated features, generated dependency folders, secrets, and documentation in one commit.
- Avoid empty or artificial commits created only to increase activity counts.
- Contribution quality, review behavior, testing, and integration work are more important than raw commit count.

11. Security and Repository Hygiene

[ ] Include .gitignore files appropriate to React, Java, IDEs, logs, build outputs, and environment files.

[ ] Commit .env.example or documented configuration templates, never real secrets.

[ ] Use synthetic data only.

[ ] Enable dependency and code scanning features where available.

[ ] Remove debug credentials, hard-coded tokens, and excessive sensitive logging before release.

[ ] Document third-party dependencies and licenses in the release documentation.

12. CI and Release Expectations

- Each service repository must run a build on pull requests using GitHub Actions.
- The shared frontend repository must run type checking, build, and available automated tests.
- A failed required CI check must be corrected before merge unless the teaching team approves an exception.
- Create at least one release candidate tag and one final release tag.
- The final release must identify compatible frontend, gateway, service, database, and infrastructure versions.
- Automated cloud deployment is encouraged but not mandatory; the deployment process must be reproducible.

13. Required Final Submission Package

Submission Item	Required Content
Repository index	GitHub organization URL and list of repositories, owners, default branches, release tags, and setup status.
Jira evidence	Project URL, final backlog/export or screenshots, sprint reports, completed issues, bugs, changes, and dependency evidence.
Project documents	Final proposal/charter, plan, SRS, architecture, ERDs, API design, test documents, guides, release notes, closure report, and retrospective.
Software release	Integrated tagged code, database scripts/migrations, Docker files, Docker Compose, configuration examples, and dependency list.
API and testing	Swagger/OpenAPI access, Postman collection/environment without secrets, automated test reports, execution report, and defect log.
Deployment	Deployed URL, deployment guide, health-check evidence, and known limitations.
Presentation and demonstration	Final presentation and live system demonstration using prepared synthetic data.
Contribution evidence	Team role table and one individual contribution statement per student with GitHub/Jira links.

14. Demonstration Preparation Checklist

[ ] Use a stable tagged release rather than uncommitted local changes.
[ ] Prepare synthetic data that demonstrates roles, business rules, success cases, and validation failures.
[ ] Demonstrate at least two cross-team end-to-end workflows.
[ ] Show Jira planning and progress evidence.
[ ] Show selected pull requests, reviews, CI checks, and contribution evidence.
[ ] Show Swagger/Postman and representative automated tests.
[ ] Explain a meaningful architecture decision and one integration problem that the teams resolved.
[ ] Prepare a fallback local environment if the cloud deployment becomes temporarily unavailable.